```
"""Convenience `elem | elem | elem ...` systax (like shell pipes).

Most elements of the model receive cars uncontrollably and inject cars into
their target element when they decide.  The Queue | Server connection however
is reversed: the server decides when to pump from the queue.  Every server
must be preceded by a queue.

To support easy construction of more complex nets with multi-connection
elements, we provide an abstraction of a Group: something with dictionaries of
named input ports and of named output ports.  Any element is a group.
Connecting two groups with a pipe actually performs a linker operation:
matching port names from one's output dictionary and another's input
dictionary are physically connected and a new group with merged dictionaries
(minus the resolved names) is returned.  Shortcut syntaxes of connecting a
group to lists, dicts, strings/nums for easily setting the names are
supported.

None is the default name for one-input elements.  Multi-input/output elements
have no default names until you set them with the `[name1, name2...] >
MultiQueue()` and `MultiInject() > [name1, name2...]` syntaxes.  Note that in
this case, unlike the `|` operator, no new group is created but the elements
themself are modified.  They heavily rely on the names for refering to
inputs/outputs in all their paramaters...

For example:
>>> print ('foo' | Inject() | 'quux') | {'quux' : 'bar'} | \
           ('bar' | Element() | 'baz')
<Group: ['foo'] -> ['baz']>

"""

def link(source, target):
    """Link source.outputs against target.inputs procuding a merged group.

    The unmatched inner source.outputs and target.intputs are discarded (they
    used to be merged but it was subtle, caused bugs and wasn't useful).  The
    group's ports are source.inputs and target.outputs.

    If one argument is a dictionary, it specifies a renaming of the
    inputs/outputs of the other group.  A list specifies renaming integers.  A
    string, int or tuple is specifies renaming None.

    The linking operation never modifies the port lists of the original groups
    (it does trigger connections between the actual elements).

    """
    import types
    # triple `#` will mark debugging prints
    ### print source, '|', target

    if type(target) in types.StringTypes + (int, tuple):
        target = {None: target}
    elif type(target) is list:
        target = dict(zip(range(len(target)), target))
    if type(target) is dict:
        outputs = source.outputs.copy()
        for old, new in target.iteritems():
            outputs[new] = source.outputs[old]
            del outputs[old]
        return Group(source.inputs, outputs)

    if type(source) in types.StringTypes + (int, tuple):
        source = {source : None}
    elif type(source) is list:
        source = dict(zip(source, range(len(source))))
    if type(source) is dict:
        inputs = target.inputs.copy()
        for new, old in source.iteritems():
            inputs[new] = target.inputs[old]
            del inputs[old]
```

```
        return Group(inputs, target.outputs)

    resolved = [name for name in source.outputs if name in target.inputs]
    for name in resolved:
        obj1, obj2 = source.outputs[name], target.inputs[name]
        ### print ' ', name, ':', obj1, '|', obj2
        if hasattr(obj1, 'connect'):
            obj1.connect(obj2)
        else:
            obj2.rconnect(obj1)
    ## inputs = source.inputs.copy()
    ## for name in target.inputs:
    ##     if name not in resolved and name not in inputs:
    ##         inputs[name] = target.inputs[name]
    ## outputs = target.outputs.copy()
    ## for name in source.outputs:
    ##     if name not in resolved and name not in outputs:
    ##         outputs[name] = source.outputs[name]
    ## return Group(inputs, outputs)
    return Group(source.inputs, target.outputs)

def merge(*groups):
    """Return merged group (no out->in linking)

    Assumes that input and output names don't collide."""

    inputs = {}
    outputs = {}
    for g in groups:
        inputs.update(g.inputs)
        outputs.update(g.outputs)
    return Group(inputs, outputs)

class Group(object):
    """Something with dictionaries of input and ouput ports.

    The values in the self.intpus and self.outputs are actual Inject/Pump
    objects that process the | operator.

    """
    def __init__(self, inputs={}, outputs={}):
        self.inputs = inputs
        self.outputs = outputs
    def __or__(self, target):  # self | target
        """link(self, target)"""
        x = link(self, target)
        ### print '  =', x
        return x
    def __ror__(self, source): # source | self
        """link(source, self)"""
        x = link(source, self)
        ### print '  =', x
        return x
    def __add__(self, other):
        """merge(self, other)"""
        return merge(self, other)
    def __str__(self):
        return "<Group: %s -> %s>" % (self.inputs.keys(), self.outputs.keys())

class Element(Group):
    """Element of single input and single output

    Initializes self.inputs and self.outputs to be {None: self}.
    Inject and Pump subclass this only for convenience.

    """
    def __init__(self):
        super(Element, self).__init__()
        self.inputs = self.outputs = {None: self}
    def __str__(self):
```

```
        """Return object.__str__(self) since Group.__str__ looks stupid."""
        return object.__str__(self)

class Inject(Element):
    """Element that injects cars into a target"""
    def connect(self, target):
        self.target = target
        return target

class Pump(Element):
    """Element that pumps cars from a source"""
    def rconnect(self, queue):
        self.queue = queue
        return self

# More complex elements take multiple inputs and outputs:

class MultiQueue(Element):
    """Sets self.inputs to refer to created slave queues

    The queues are created by self.city.make_queue to allow selection.

    This is asymmetric to MultiInject in that it doesn't create pumps since
    that would be redundant - every pump always needs an associated queue...

    Note that this wins when also inheriting from Pump.

    """
    def __lt__(self, names):            # names > self (also self < names)
        """Creates queues and attaches them to input names; return self"""
        self.inputs = dict([(name, self.city.make_queue()) for name in names])
        return self

class MultiInject(Element):
    """Sets self.injects to refer to created slave injects

    Note that this wins when also inheriting from Inject.

    """
    def __gt__(self, names):            # self > names (also names < self)
        """Creates injects and attaches them to output names; return self"""
        self.outputs = dict([(name, Inject()) for name in names])
        return self
```